introduces some security issues, which we'll get to later in this section. You should be aware that this mechanism is deprecated in PHP 8.4 and is set to disappear completely in PHP 9 (I wasn't kidding about the security risks). Returning to the `cli-server` container, we can see how PHP has serialized data in the session file:

```
root@0d72be74121b:/usr/src/myapp# cat /tmp/sess_3fe1329aa9a3575373814e
bf8aa59857
visits|i:2;
```

Session Cookie Management

The eagle-eyed amongst you hopefully spotted something in the first example of this section: the `PHPSESSID` cookie that was set when we initialized the session with `session_start()` (and did not pass a `PHPSESSID` cookie with the request) had no expiry metadata, which means it will persist only until the client session terminates (when you close the browser). Session cookie parameters are configurable with the following ini directives:

Table 11-5. *PHP session `ini` directives*

Directive	Default	
`session.cookie_lifetime`	`0`	Expiration timestamp relative to server time
`session.cookie_path`	`"/"`	The path value of the cookie
`session.cookie_domain`	`""`	The domain
`session.cookie_secure`	`0`	If set to `1`, only send cookie with HTTPS scheme
`session.cookie_httponly`	`0`	Only send with HTTP scheme
`session.cookie_samesite`	`""`	Set `SameSite` attribute

You can view the current parameters during script execution using the `session_get_cookie_params()` function, and you can set them on-the-fly with `session_set_cookie_params()` and pass an associative array of options and values – `session_set_cookie_params(['lifetime' => 3600]);` would trigger the addition of `Max-Age` and `Expires` parameters to the `Set-Cookie:` response header. Be sure to call `session_set_cookie_params()` *before* the session is started.